
Phenotypic Prediction from Genotypic Data using a complex Auto-encoder+Resnet18

Harsh Patidar

1 MOTIVATION

Soybean breeders and geneticists face a major challenge: predicting complex phenotypic traits from extremely high-dimensional genomic data. A single soybean genotype can contain tens of thousands of SNP markers (in this case 66,000), making traditional machine-learning models slow, memory-intensive, and often inaccurate due to noise and redundancy in the data.

This motivated the development of a novel approach that:

Compresses extremely high-dimensional SNP data using an Autoencoder, reducing 66k markers to a small latent space (100–500 features) with minimal accuracy loss (from 0.732 to 0.722 PCC).

2 LIBRARIES USED IN THE PROJECT

- **Pandas:** Used for efficient data loading, cleaning, and manipulation of genotype and phenotype tables.
- **NumPy:** Provides fast numerical operations, array handling, and mathematical computations essential for preprocessing.
- **Scikit-learn:** Offers tools for preprocessing (e.g., `StandardScaler`), cross-validation utilities such as `KFold`, and evaluation metrics including R^2 , MSE, and MAE.
- **PyTorch:** Core deep learning framework used for designing and training the Autoencoder and ResNet models. Includes:

- **torch.nn:** Provides neural network layers such as Linear, BatchNorm1d, Dropout, and activation functions.
- **torch.optim:** Contains optimization algorithms such as Adam and SGD.
- **torch.nn.functional:** Supports functional API operations such as activations and loss functions.
- **Matplotlib:** Used for visualization of training curves, loss plots, and model performance metrics.
- **SciPy:** Provides statistical functions, including computation of correlation metrics such as Pearson's Correlation Coefficient (PCC).

3 STRUCTURE OF AUTO-ENCODER

Listing 1: Autoencoder Architecture

```
self.encoder = nn.Sequential(
    nn.Linear(input_dim, 2048),
    nn.BatchNorm1d(2048),
    nn.LeakyReLU(),
    nn.Dropout(0.3),

    nn.Linear(2048, 1024),
    nn.BatchNorm1d(1024),
    nn.LeakyReLU(),
    nn.Dropout(0.3),

    nn.Linear(1024, 512),
    nn.BatchNorm1d(512),
    nn.LeakyReLU(),
    nn.Dropout(0.3),

    nn.Linear(512, 256),
    nn.BatchNorm1d(256),
    nn.LeakyReLU(),
    nn.Dropout(0.3),

    nn.Linear(256, encoding_dim),
    nn.Sigmoid()
)

# Decoder
self.decoder = nn.Sequential(
    nn.Linear(encoding_dim, 256),
```

```

nn.BatchNorm1d(256),
nn.LeakyReLU(),

nn.Linear(256, 512),
nn.BatchNorm1d(512),
nn.LeakyReLU(),
nn.Dropout(0.3),

nn.Linear(512, 1024),
nn.BatchNorm1d(1024),
nn.LeakyReLU(),
nn.Dropout(0.3),

nn.Linear(1024, 2048),
nn.BatchNorm1d(2048),
nn.LeakyReLU(),
nn.Dropout(0.3),

nn.Linear(2048, input_dim),
nn.Sigmoid()
)

```

3.1 ROLE OF EACH LAYER

1. **Linear Layer:** Performs a linear transformation of the input using a learnable weight matrix W and bias b . The output is computed as:

$$y = Wx + b$$

This layer extracts and combines features from the high-dimensional SNP vector.

2. **BatchNorm1d:** Normalizes the activations so that each neuron's output has approximately zero mean and unit variance. This stabilizes training, accelerates convergence, and reduces internal covariate shift.
3. **LeakyReLU:** A nonlinear activation function that allows a small gradient when the input is negative. This prevents neurons from “dying” and helps the network capture complex, nonlinear genomic patterns.
4. **Dropout:** A regularization technique that randomly switches off a fraction of neurons during training. This forces the model to learn more robust representations and prevents overfitting.
5. **Sigmoid (Final Layer):** Applied after the last linear layer of the decoder to map outputs into the range $[0, 1]$. Since SNP data is often encoded as normalized values (0, 1, or intermediate imputed values), the sigmoid ensures that the reconstructed output matches the original input scale of the genotype matrix.

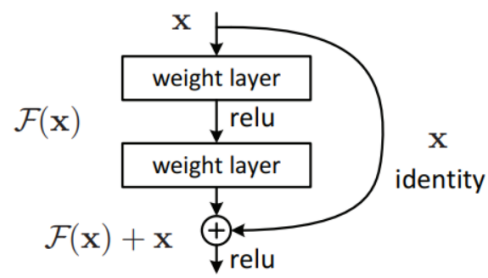


Figure 4.1: Skip Connections Diagram

4 MAGIC OF RESNET

In order to solve the problem of the vanishing/exploding gradient, this architecture introduced the concept called Residual Blocks. In this network, we use a technique called skip connections. The skip connection connects activations of a layer to further layers by skipping some layers in between. This forms a residual block. Resnets are made by stacking these residual blocks together.